

Module 5 Notes: Counting, Summing, and Finding Maximum/Minimum in a Linked List

1. Current progress analysis

You are ready for Module 5 because you already understand the three foundations that this module depends on.

From Module 1

You learned why linked lists exist.

Arrays are fast when we already know the index, but arrays have a fixed size after creation. Linked lists are more flexible because they can grow and shrink one node at a time.

Basic tradeoff:

Structure	Strength	Weakness
Array	Fast direct access by index	Fixed size, shifting needed for insertion/deletion
Linked list	Flexible size, easier pointer-based insertion/deletion	No direct jumping to an index

From Module 2

You learned what a node is.

A singly linked-list node has:

[data | next]

Meaning:

Part	Meaning
data	The actual value stored in the node
next	Address of the next node

C node definition:

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

You also learned the key pointer movement:

```
p = p->next;
```

This moves pointer `p` to the next node.

From Module 3

You learned traversal, display, and list creation.

Traversal means visiting nodes one by one:

```
first -> [8] -> [3] -> [7] -> [12] -> NULL
```

Basic display pattern:

```
void display(struct Node *p) {  
    while (p != NULL) {  
        printf("%d ", p->data);  
        p = p->next;  
    }  
}
```

Important rule:

Do not move `first` during traversal. Use a temporary pointer like `p`.

From Module 4

You learned recursion on linked lists.

Recursive functions usually have this form:

```
if (p == NULL) {  
    return some_base_value;  
}
```

```
return something_using(p->data, recursive_function(p->next));
```

The base case is usually:

```
p == NULL
```

That means the list has ended.

Module 5 uses both ideas:

1. Iterative traversal using `while`.
2. Recursive traversal using function calls.

2. Big idea of Module 5

Module 5 teaches how to calculate useful values from a linked list.

Given this linked list:

```
8 -> 3 -> 7 -> 12 -> NULL
```

We may want to answer questions like:

How many nodes are there?	4
What is the sum of all values?	30
What is the maximum value?	12
What is the minimum value?	3

The important idea is:

Count, sum, maximum, and minimum are all traversal problems.

That means we must visit nodes one by one.

The common pattern is:

```
while (p != NULL) {  
    // use p->data somehow
```

```
p = p->next;  
}
```

Only the work inside the loop changes.

Operation	What we do at each node
Count	Add 1 to a counter
Sum	Add <code>p->data</code> to a total
Maximum	Check if <code>p->data</code> is bigger than current max
Minimum	Check if <code>p->data</code> is smaller than current min

3. Counting nodes

Counting means finding how many nodes are in the linked list.

Example:

```
8 -> 3 -> 7 -> 12 -> NULL
```

There are 4 nodes.

Iterative count idea

Start with:

```
c = 0
```

Then visit each node.

For every real node, increase `c` by 1.

Stop when `p` becomes `NULL`.

Iterative count code

```
int count(struct Node *p) {  
    int c = 0;  
  
    while (p != NULL) {
```

```
        c++;  
        p = p->next;  
    }  
  
    return c;  
}
```

Meaning of each line

```
int count(struct Node *p)
```

The function receives a pointer to the first node of the list.

```
int c = 0;
```

`c` stores the number of nodes counted so far.

```
while (p != NULL)
```

Keep looping while `p` points to a real node.

```
c++;
```

Count the current node.

```
p = p->next;
```

Move to the next node.

```
return c;
```

Return the final number of nodes.

Dry run: iterative count

List:

```
8 -> 3 -> 7 -> 12 -> NULL
```

Start:

```
c = 0  
p points to 8
```

Step	p points to	Action	c
1	8	Count node	1
2	3	Count node	2
3	7	Count node	3
4	12	Count node	4
5	NULL	Stop	4

Final answer:

```
4
```

Empty list case

If the list is empty:

```
first = NULL
```

Then p is already NULL.

The loop does not run.

So the function returns:

```
0
```

That is correct because an empty list has zero nodes.

4. Recursive count

Recursive count solves the same problem using recursion instead of a loop.

Recursive thinking

For this list:

```
8 -> 3 -> 7 -> 12 -> NULL
```

Think like this:

```
count(8 -> 3 -> 7 -> 12) = 1 + count(3 -> 7 -> 12)
count(3 -> 7 -> 12)      = 1 + count(7 -> 12)
count(7 -> 12)           = 1 + count(12)
count(12)                = 1 + count(NULL)
count(NULL)              = 0
```

The current node contributes `1`.

The rest of the answer comes from the remaining list.

Recursive count code

```
int RCount(struct Node *p) {
    if (p == NULL) {
        return 0;
    }

    return RCount(p->next) + 1;
}
```

Base case

```
if (p == NULL) {
    return 0;
}
```

This means:

If there is no node, the count is zero.

Recursive case

```
return RCount(p->next) + 1;
```

This means:

Count the rest of the list, then add 1 for the current node.

Dry run: recursive count

List:

8 -> 3 -> 7 -> 12 -> NULL

Trace:

```
RCount(8)
= RCount(3) + 1
= RCount(7) + 1 + 1
= RCount(12) + 1 + 1 + 1
= RCount(NULL) + 1 + 1 + 1 + 1
= 0 + 1 + 1 + 1 + 1
= 4
```

Final answer:

4

Alternative recursive count version

Because `return` stops the function immediately, we can remove the `else`:

```
int RCount(struct Node *p) {
    if (p == NULL) {
        return 0;
    }

    return RCount(p->next) + 1;
}
```

This is usually cleaner than:

```
int RCount(struct Node *p) {
    if (p == NULL) {
        return 0;
    } else {
```



```
        return RCount(p->next) + 1;
    }
}
```

Both are correct.

Recursive count with a local variable

Some versions use a local variable:

```
int RCount(struct Node *p) {
    int x = 0;

    if (p != NULL) {
        x = RCount(p->next);
        return x + 1;
    }

    return 0;
}
```

Meaning:

1. First count the rest of the list.
2. Store that result in `x`.
3. Add 1 for the current node.

5. Summing node values

Sum means adding all values stored in the nodes.

Example:

```
8 -> 3 -> 7 -> 12 -> NULL
```

Sum:

```
8 + 3 + 7 + 12 = 30
```

Iterative sum idea

Start with:

```
total = 0
```

Then visit each node.

At every node, add the node's data to `total`.

Iterative sum code

```
int sum(struct Node *p) {  
    int total = 0;  
  
    while (p != NULL) {  
        total = total + p->data;  
        p = p->next;  
    }  
  
    return total;  
}
```

Shorter version:

```
int sum(struct Node *p) {  
    int total = 0;  
  
    while (p != NULL) {  
        total += p->data;  
        p = p->next;  
    }  
  
    return total;  
}
```

Both versions mean the same thing.

Meaning of each line

```
int total = 0;
```

The sum starts at zero.

```
total += p->data;
```

Add the current node's value to the running total.

```
p = p->next;
```

Move to the next node.

```
return total;
```

Return the final sum.

Dry run: iterative sum

List:

```
8 -> 3 -> 7 -> 12 -> NULL
```

Start:

```
total = 0  
p points to 8
```

Step	p points to	Action	total
1	8	Add 8	8
2	3	Add 3	11
3	7	Add 7	18
4	12	Add 12	30
5	NULL	Stop	30

Final answer:

```
30
```

Empty list case

If the list is empty, the loop does not run.

The function returns:

0

That is correct because the sum of no values is zero.

6. Recursive sum

Recursive sum adds the current node's data plus the sum of the remaining list.

Recursive thinking

For this list:

8 -> 3 -> 7 -> 12 -> NULL

Think:

```
RSum(8)    = RSum(3) + 8
RSum(3)    = RSum(7) + 3
RSum(7)    = RSum(12) + 7
RSum(12)   = RSum(NULL) + 12
RSum(NULL) = 0
```

Recursive sum code

```
int RSum(struct Node *p) {
    if (p == NULL) {
        return 0;
    }

    return RSum(p->next) + p->data;
}
```

Base case

```
if (p == NULL) {
    return 0;
}
```

An empty list contributes zero to the sum.

Recursive case

```
return RSum(p->next) + p->data;
```

This means:

1. Find the sum of the remaining list.
2. Add the current node's data.

Dry run: recursive sum

List:

```
8 -> 3 -> 7 -> 12 -> NULL
```

Trace:

```
RSum(8)
= RSum(3) + 8
= RSum(7) + 3 + 8
= RSum(12) + 7 + 3 + 8
= RSum(NULL) + 12 + 7 + 3 + 8
= 0 + 12 + 7 + 3 + 8
= 30
```

Final answer:

```
30
```

Count and sum comparison

Count:

```
return RCount(p->next) + 1;
```

Sum:

```
return RSum(p->next) + p->data;
```

Difference:

Function	Adds what?
Count	Adds <code>1</code> for every node
Sum	Adds <code>p->data</code> for every node

7. Finding the maximum value

Maximum means finding the largest value in the linked list.

Example:

```
8 -> 3 -> 7 -> 12 -> 9 -> NULL
```

Maximum:

```
12
```

Important initialization rule

Do not always start maximum with `0`.

This is wrong for lists with only negative numbers.

Example:

```
-8 -> -3 -> -10 -> NULL
```

The maximum is:

```
-3
```

But if we start with:

```
int m = 0;
```

The function may incorrectly return `0`, even though `0` is not in the list.

So we need to start with the smallest possible value.

In C, include:

```
#include <stdint.h>
```

Then use:

```
INT32_MIN
```

Example:

```
int m = INT32_MIN;
```

This means:

Start with a value so small that any real list value should be bigger.

Iterative maximum code

```
#include <stdint.h>

int max(struct Node *p) {
    int m = INT32_MIN;

    while (p != NULL) {
        if (p->data > m) {
            m = p->data;
        }

        p = p->next;
    }

    return m;
}
```

Meaning of each line

```
int m = INT32_MIN;
```

Start `m` with the smallest possible 32-bit integer value.

```
if (p->data > m)
```

Check whether the current node's data is bigger than the current maximum.

```
m = p->data;
```

If it is bigger, update the maximum.

```
p = p->next;
```

Move to the next node.

Dry run: iterative maximum

List:

```
8 -> 3 -> 7 -> 12 -> 9 -> NULL
```

Start:

```
m = very small number
```

Step	p points to	Comparison	m
1	8	8 is bigger than very small value	8
2	3	3 is not bigger than 8	8
3	7	7 is not bigger than 8	8
4	12	12 is bigger than 8	12
5	9	9 is not bigger than 12	12
6	NULL	Stop	12

Final answer:

```
12
```


Another maximum dry run

List:

```
3 -> 5 -> 7 -> 32 -> 15 -> 2 -> 8 -> 20 -> NULL
```

Start:

```
m = INT32_MIN
```

p points to	m after checking
3	3
5	5
7	7
32	32
15	32
2	32
8	32
20	32

Final maximum:

```
32
```

8. Recursive maximum

Recursive maximum is a little harder than recursive count and sum.

The idea is:

```
Find the maximum of the rest of the list.  
Then compare it with the current node.
```

For this list:

8 -> 3 -> 7 -> 12 -> 9 -> NULL

We ask:

Maximum of entire list = larger of:
1. current node value
2. maximum of remaining list

Recursive maximum code

```
#include <stdint.h>

int RMax(struct Node *p) {
    int x;

    if (p == NULL) {
        return INT32_MIN;
    }

    x = RMax(p->next);

    if (x > p->data) {
        return x;
    } else {
        return p->data;
    }
}
```

Meaning of the code

```
if (p == NULL) {
    return INT32_MIN;
}
```

If the list is empty at this point, return the smallest possible value.

This lets any real node value become larger than the base value.

```
x = RMax(p->next);
```

Find the maximum value in the rest of the list.

```

if (x > p->data) {
    return x;
} else {
    return p->data;
}

```

Return whichever value is larger:

1. `x`, the maximum of the rest of the list.
2. `p->data`, the current node's value.

Shorter recursive maximum using ternary operator

```

#include <stdint.h>

int RMax(struct Node *p) {
    int x;

    if (p == NULL) {
        return INT32_MIN;
    }

    x = RMax(p->next);

    return x > p->data ? x : p->data;
}

```

This line:

```

return x > p->data ? x : p->data;

```

Means:

```

If x is bigger, return x.
Otherwise, return p->data.

```

Dry run: recursive maximum

List:

```

8 -> 3 -> 7 -> 12 -> 9 -> NULL

```

Calling phase:

```
RMax(8)
RMax(3)
RMax(7)
RMax(12)
RMax(9)
RMax(NULL)
```

Base case:

```
RMax(NULL) returns INT32_MIN
```

Returning phase:

Returning from	Compare	Return
RMax(9)	max(INT32_MIN, 9)	9
RMax(12)	max(9, 12)	12
RMax(7)	max(12, 7)	12
RMax(3)	max(12, 3)	12
RMax(8)	max(12, 8)	12

Final answer:

```
12
```

9. Finding the minimum value

Minimum means finding the smallest value in the linked list.

Example:

```
8 -> 3 -> 7 -> 12 -> 9 -> NULL
```

Minimum:

3

Minimum is the opposite of maximum.

For maximum:

```
if (p->data > m) {  
    m = p->data;  
}
```

For minimum:

```
if (p->data < m) {  
    m = p->data;  
}
```

Important initialization rule

For minimum, start with the largest possible value.

In C:

```
#include <stdint.h>
```

Then use:

```
INT32_MAX
```

Example:

```
int m = INT32_MAX;
```

This means:

Start with a value so large that any real list value should be smaller.

Iterative minimum code

```
#include <stdint.h>

int min(struct Node *p) {
    int m = INT32_MAX;

    while (p != NULL) {
        if (p->data < m) {
            m = p->data;
        }

        p = p->next;
    }

    return m;
}
```

Dry run: iterative minimum

List:

8 -> 3 -> 7 -> 12 -> 9 -> NULL

Start:

m = very large number

Step	p points to	Comparison	m
1	8	8 is smaller than very large value	8
2	3	3 is smaller than 8	3
3	7	7 is not smaller than 3	3
4	12	12 is not smaller than 3	3
5	9	9 is not smaller than 3	3
6	NULL	Stop	3

Final answer:

10. Recursive minimum

Recursive minimum works like recursive maximum, but the comparison is reversed.

Recursive minimum code

```
#include <stdint.h>

int RMin(struct Node *p) {
    int x;

    if (p == NULL) {
        return INT32_MAX;
    }

    x = RMin(p->next);

    return x < p->data ? x : p->data;
}
```

Meaning of the code

```
if (p == NULL) {
    return INT32_MAX;
}
```

If the list is empty at this point, return the largest possible value.

This lets any real node value become smaller than the base value.

```
x = RMin(p->next);
```

Find the minimum value in the rest of the list.

```
return x < p->data ? x : p->data;
```

Return whichever value is smaller:

1. `x`, the minimum of the rest of the list.
2. `p->data`, the current node's value.

Dry run: recursive minimum

List:

```
8 -> 3 -> 7 -> 12 -> 9 -> NULL
```

Calling phase:

```
RMin(8)
RMin(3)
RMin(7)
RMin(12)
RMin(9)
RMin(NULL)
```

Base case:

```
RMin(NULL) returns INT32_MAX
```

Returning phase:

Returning from	Compare	Return
<code>RMin(9)</code>	<code>min(INT32_MAX, 9)</code>	9
<code>RMin(12)</code>	<code>min(9, 12)</code>	9
<code>RMin(7)</code>	<code>min(9, 7)</code>	7
<code>RMin(3)</code>	<code>min(7, 3)</code>	3
<code>RMin(8)</code>	<code>min(3, 8)</code>	3

Final answer:

```
3
```


11. Why count, sum, max, and min cannot stop early

For a normal unsorted linked list, these operations usually need the whole list.

Count cannot stop early

To know how many nodes exist, we must visit every node.

Example:

```
8 -> 3 -> 7 -> 12 -> NULL
```

If we stop at `7`, we do not know that `12` still exists.

Sum cannot stop early

To calculate the total, every value matters.

Example:

```
8 -> 3 -> 7 -> 12 -> NULL
```

If we stop at `7`, we miss `12`, so the sum is wrong.

Maximum cannot stop early in a general list

Example:

```
100 -> 2 -> 3 -> 500 -> NULL
```

After seeing `100`, we still cannot stop because a bigger value may appear later.

Minimum cannot stop early in a general list

Example:

```
-10 -> 5 -> 8 -> -50 -> NULL
```

After seeing `-10`, we still cannot stop because a smaller value may appear later.

12. Full C program for Module 5

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *first = NULL;

void create(int A[], int n) {
    if (n <= 0) {
        first = NULL;
        return;
    }

    struct Node *t;
    struct Node *last;

    first = (struct Node *)malloc(sizeof(struct Node));
    first->data = A[0];
    first->next = NULL;
    last = first;

    for (int i = 1; i < n; i++) {
        t = (struct Node *)malloc(sizeof(struct Node));
        t->data = A[i];
        t->next = NULL;
        last->next = t;
        last = t;
    }
}

void display(struct Node *p) {
    while (p != NULL) {
        printf("%d ", p->data);
        p = p->next;
    }
}

int count(struct Node *p) {
    int c = 0;
```

```

        while (p != NULL) {
            c++;
            p = p->next;
        }

        return c;
    }

    int RCount(struct Node *p) {
        if (p == NULL) {
            return 0;
        }

        return RCount(p->next) + 1;
    }

    int sum(struct Node *p) {
        int total = 0;

        while (p != NULL) {
            total += p->data;
            p = p->next;
        }

        return total;
    }

    int RSum(struct Node *p) {
        if (p == NULL) {
            return 0;
        }

        return RSum(p->next) + p->data;
    }

    int max(struct Node *p) {
        int m = INT32_MIN;

        while (p != NULL) {
            if (p->data > m) {
                m = p->data;
            }

            p = p->next;
        }

        return m;
    }
}

```

```

int RMax(struct Node *p) {
    int x;

    if (p == NULL) {
        return INT32_MIN;
    }

    x = RMax(p->next);

    return x > p->data ? x : p->data;
}

int min(struct Node *p) {
    int m = INT32_MAX;

    while (p != NULL) {
        if (p->data < m) {
            m = p->data;
        }

        p = p->next;
    }

    return m;
}

int RMin(struct Node *p) {
    int x;

    if (p == NULL) {
        return INT32_MAX;
    }

    x = RMin(p->next);

    return x < p->data ? x : p->data;
}

int main() {
    int A[] = {8, 3, 7, 12, 9};

    create(A, 5);

    printf("List: ");
    display(first);

    printf("\n\nCount: %d", count(first));
}

```

```
    printf("\nRecursive Count: %d", RCount(first));

    printf("\n\nSum: %d", sum(first));
    printf("\nRecursive Sum: %d", RSum(first));

    printf("\n\nMax: %d", max(first));
    printf("\nRecursive Max: %d", RMax(first));

    printf("\n\nMin: %d", min(first));
    printf("\nRecursive Min: %d", RMin(first));

    return 0;
}
```

Expected output:

```
List: 8 3 7 12 9

Count: 5
Recursive Count: 5

Sum: 39
Recursive Sum: 39

Max: 12
Recursive Max: 12

Min: 3
Recursive Min: 3
```

13. Complexity analysis

Let n be the number of nodes in the linked list.

Time complexity

Count, sum, max, and min all visit every node.

So their time complexity is:

```
O(n)
```

Why?

Because if the list has 5 nodes, the function may visit 5 nodes.

If the list has 5000 nodes, the function may visit 5000 nodes.

The work grows with the number of nodes.

Space complexity: iterative versions

Iterative versions use only a few variables:

```
p  
c  
total  
m
```

The number of extra variables does not grow with the list size.

So iterative extra space is:

```
O(1)
```

Space complexity: recursive versions

Recursive versions create one function call for each node, plus one final call for `NULL`.

For `n` nodes, recursive functions make about:

```
n + 1 calls
```

These calls are stored on the system stack.

So recursive extra space is:

```
O(n)
```

Complexity table

Operation	Iterative Time	Iterative Extra Space	Recursive Time	Recursive Extra Space
Count	$O(n)$	$O(1)$	$O(n)$	$O(n)$
Sum	$O(n)$	$O(1)$	$O(n)$	$O(n)$

Operation	Iterative Time	Iterative Extra Space	Recursive Time	Recursive Extra Space
Maximum	O(n)	O(1)	O(n)	O(n)
Minimum	O(n)	O(1)	O(n)	O(n)

14. Common beginner mistakes

Mistake 1: Moving `first` directly

Wrong:

```
while (first != NULL) {
    first = first->next;
}
```

Problem:

This moves the main pointer away from the first node.

You may lose access to the original beginning of the list.

Correct:

```
struct Node *p = first;

while (p != NULL) {
    p = p->next;
}
```

Move a temporary pointer, not `first`.

Mistake 2: Starting maximum with `0`

Wrong:

```
int m = 0;
```

Problem:

This fails if all list values are negative.

Example:

```
-8 -> -3 -> -10 -> NULL
```

The correct maximum is:

```
-3
```

But starting with `0` may incorrectly return:

```
0
```

Better:

```
int m = INT32_MIN;
```

Mistake 3: Starting minimum with `0`

Wrong:

```
int m = 0;
```

Problem:

This can fail if all list values are positive.

Example:

```
8 -> 3 -> 7 -> NULL
```

The correct minimum is:

```
3
```

Better:

```
int m = INT32_MAX;
```


Mistake 4: Forgetting the recursive base case

Wrong:

```
int RCount(struct Node *p) {  
    return RCount(p->next) + 1;  
}
```

Problem:

When `p` becomes `NULL`, the function tries to access:

```
p->next
```

That is invalid.

Correct:

```
int RCount(struct Node *p) {  
    if (p == NULL) {  
        return 0;  
    }  
  
    return RCount(p->next) + 1;  
}
```

Mistake 5: Using `p->data` when `p` is `NULL`

Wrong:

```
while (p != NULL) {  
    p = p->next;  
    printf("%d", p->data);  
}
```

Problem:

After moving `p`, it may become `NULL`.

Then `p->data` is invalid.

Correct:

```
while (p != NULL) {
    printf("%d", p->data);
    p = p->next;
}
```

Use the node first, then move.

Mistake 6: Thinking recursion is always better

Recursive code can look clean, but it uses stack memory.

For very large lists, iterative code is usually safer because it uses only $O(1)$ extra space.

15. Key patterns to memorize

Traversal pattern

```
while (p != NULL) {
    // work with p->data
    p = p->next;
}
```

Count pattern

```
int c = 0;

while (p != NULL) {
    c++;
    p = p->next;
}
```

Sum pattern

```
int total = 0;

while (p != NULL) {
    total += p->data;
    p = p->next;
}
```

Maximum pattern

```
int m = INT32_MIN;

while (p != NULL) {
    if (p->data > m) {
        m = p->data;
    }
    p = p->next;
}
```

Minimum pattern

```
int m = INT32_MAX;

while (p != NULL) {
    if (p->data < m) {
        m = p->data;
    }
    p = p->next;
}
```

Recursive count pattern

```
if (p == NULL) return 0;
return RCount(p->next) + 1;
```

Recursive sum pattern

```
if (p == NULL) return 0;
return RSum(p->next) + p->data;
```

Recursive maximum pattern

```
if (p == NULL) return INT32_MIN;
x = RMax(p->next);
return x > p->data ? x : p->data;
```

Recursive minimum pattern

```
if (p == NULL) return INT32_MAX;  
x = RMin(p->next);  
return x < p->data ? x : p->data;
```

16. Practice dry run

Given list:

10 -> 20 -> 5 -> 40 -> NULL

Count

There are 4 nodes.

Count = 4

Sum

10 + 20 + 5 + 40 = 75

Sum = 75

Maximum

Largest value:

Max = 40

Minimum

Smallest value:

Min = 5

17. More practice questions

Question 1

Given:

```
2 -> 4 -> 6 -> 8 -> NULL
```

What is the count?

Answer:

```
4
```

Question 2

Given:

```
2 -> 4 -> 6 -> 8 -> NULL
```

What is the sum?

Answer:

```
20
```

Question 3

Given:

```
2 -> 4 -> 6 -> 8 -> NULL
```

What is the maximum?

Answer:

```
8
```

Question 4

Given:

2 -> 4 -> 6 -> 8 -> NULL

What is the minimum?

Answer:

2

Question 5

Why should maximum not start with 0?

Answer:

Because the list may contain only negative numbers. If maximum starts with 0, the function may return 0 even when 0 is not in the list.

Question 6

Why should minimum not start with 0?

Answer:

Because the list may contain only positive numbers. Starting with 0 may incorrectly keep 0 as the minimum.

Question 7

Why do recursive versions use $O(n)$ space?

Answer:

Because each recursive call is stored on the system stack. For n nodes, there are about $n + 1$ calls.

Question 8

Why do count, sum, max, and min take $O(n)$ time?

Answer:

Because each operation may need to visit every node in the list.

18. Final Module 5 summary

Module 5 teaches how to calculate values from a linked list.

The four main operations are:

1. Count nodes.
2. Sum node values.
3. Find the maximum value.
4. Find the minimum value.

The main idea is:

Module 5 is traversal plus calculation.

The common traversal structure is:

```
while (p != NULL) {  
    // use p->data  
    p = p->next;  
}
```

For count:

```
c++;
```

For sum:

```
total += p->data;
```

For maximum:

```
if (p->data > m) {  
    m = p->data;  
}
```

For minimum:

```
if (p->data < m) {
    m = p->data;
}
```

Recursive versions use the same idea from Module 4:

```
if (p == NULL) {
    return base_value;
}

return answer_using(function(p->next), p->data);
```

Important initialization rules:

Operation	Starting value
Count	0
Sum	0
Maximum	INT32_MIN
Minimum	INT32_MAX

Important complexity rules:

Version	Time	Extra Space
Iterative	$O(n)$	$O(1)$
Recursive	$O(n)$	$O(n)$

The most important lesson:

Once you understand traversal, many linked-list operations become simple. You just change what you do at each node.